TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

NATIONAL COLLEGE OF ENGINEERING

A MINOR PROJECT REPORT

ON

A UNIFIED AI TEXT DETECTION, PLAGIARISM ANALYSIS AND DOCUMENT FORENSICS SYSTEM

SUBMITTED BY:

AAVASH TIWARI (NCE080BCT001)

BIJAY NATH SHRESTHA (NCE080BCT009)

RAMESHWOR POUDEL (NCE080BCT028)

SUDIP DHUNGANA (NCE080BCT041)

SUBMITTED TO:

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

AUGUST, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled **"A Unified AI Text Detection, Plagiarism Analysis and Document Forensics System"** submitted by **Aavash Tiwari**, **Bijay Nath Shrestha**, **Rameshwor Poudel**, **Sudip Dhungana** in partial fulfillment of the requirements for the Bachelor's degree in Electronics, Information and Communication Engineering & Computer Engineering.

.....
Supervisor
Mrs. Sharmila Bista
Department of Electronics and Computer Engineering,
National College of Engineering, IOE, TU.

.....
External Examiner
Full Name
Designation
Affiliation

Date of approval:

Copyright

Acknowledgement

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering at National College of Engineering for providing the resources, facilities, and academic environment that made this minor project possible. The department's continuous support and encouragement played a significant role throughout the planning and development of our work.

We are especially grateful to our project supervisor, **Mrs. Sharmila Bista**, for her invaluable guidance, constructive feedback, and constant encouragement during every phase of this project. Her expertise, patience, and insightful suggestions helped us overcome numerous technical challenges and greatly improved the quality of our work.

Finally, we extend our heartfelt gratitude to our families, friends, and classmates for their unwavering support, motivation, and understanding throughout this journey. Their encouragement inspired us to remain committed to our goals.

Aavash Tiwari	NCE080BCT001
Bijay Nath Shrestha	NCE080BCT009
Rameshwor Poudel	NCE080BCT028
Sudip Dhungana	NCE080BCT041

Abstract

Academic integrity is increasingly challenged by the accessibility of generative AI tools capable of producing sophisticated, human-like text, rendering traditional plagiarism detection methods, which rely on direct string matching, largely ineffective. This project presents the design and development of an integrated web-based system for Generative AI Text Detection, Plagiarism Analysis, and Document Forensics, aimed at providing educators with a reliable mechanism to verify the originality and authenticity of academic submissions. The system employs a supervised machine learning pipeline, using TF-IDF feature extraction and a Multinomial Naïve Bayes classifier trained on the Kaggle DAIGT v2 dataset, to distinguish between human-authored and AI-generated content. A source verification module, built on the Firecrawl Search API, performs similarity comparison against web-scraped content to detect paraphrased or restructured plagiarism, while a forensic analysis module inspects document metadata to identify tampering. The platform is built on a decoupled architecture, with a React-based frontend providing role-based dashboards for Students, Teachers, and Django Administrators, and a Django backend orchestrating authentication, data storage, and inference. As of the mid-term stage, the classifier has achieved an accuracy of 92%, a precision of 0.89, and an F1-score of 0.90 on the detection task, and the core authentication, dashboard, and backend modules have been implemented. Remaining work includes integrating live analysis APIs into the frontend, extending forensic and OCR/HTR capabilities, and deploying the system with containerized infrastructure. The completed system is expected to offer a scalable, accurate, and transparent tool for upholding academic integrity in educational institutions.

Keywords: *plagiarism detection, generative AI text detection, document forensics, TF-IDF, Naive Bayes, academic integrity*

Contents

Page of Approval ii

Copyright iii

Acknowledgements iv

Abstract v

List of Figures ix

List of Tables x

List of Abbreviations xi

1. Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.3.1 General Objective 2

1.3.2 Specific Objectives 2

1.4 Scope 2

2. Literature Review 4

2.1 Related Work 4

2.2 Related Theory 4

2.2.1 Stylometric Analysis 4

2.2.2 Neural Networks for Recognition (OCR and HTR) 5

2.2.3 Mathematical Foundation 5

2.2.4 Metadata Forensic Analysis 6

3. Methodology 7

3.1 Time Schedule 7

3.2 Implementation detail 8

3.2.1 Frontend Development 8

3.2.2 Backend Development 8

3.2.3 Generative AI Text Detection Module 8
3.2.4 OCR and Document Extraction Module 9
3.2.5 Submission Similarity and Metadata Forensics 9
3.2.6 Deployment Strategy 9

4. System Design **12**
4.1 Requirement Analysis 12
4.1.1 Functional Requirements 12
4.1.2 Non-Functional Requirements 13
4.2 Website Development for End Users 14
4.2.1 System Architecture 14
4.2.2 User Authentication and Security 15
4.2.3 Functional Workflow 15
4.3 Proposed System Design 17
4.3.1 Use Case Diagram 17
4.3.2 Data Flow Diagram 18
4.3.3 System Flowchart 21
4.3.4 Data Preparation and Machine Learning Workflow 22

5. Results and Discussion **25**
5.1 Experiment Setup 25
5.1.1 Dataset and Preprocessing 25
5.1.2 Feature Extraction 25
5.2 Software Development Approach 25
5.2.1 Frontend Implementation 25
5.2.2 Backend Architecture 26
5.2.3 API Development 26
5.3 Model Deployment 26
5.3.1 AI Text Detection Model 26
5.3.2 Optical Character Recognition Integration 26
5.3.3 Plagiarism Verification Pipeline 26
5.4 Testing and Evaluation 27
5.4.1 Classifier Performance 27
5.4.2 OCR Accuracy 27
5.4.3 System Load Testing 27
5.4.4 User Acceptance 27

6. Conclusion **28**

7. Limitations and Future Enhancements **29**

7.1 Limitations 29
7.2 Future Enhancements 29

Appendices **31**

References **31**

List of Figures

3.1 Gantt chart showing the project timeline 7
3.2 Evaluation metrics. 10
3.3 Evaluation metrics 11

4.1 Use Case Diagram of the System 17
4.2 Level 0 Data Flow Diagram of the System 18
4.3 Level 1 Data Flow Diagram of the System 20
4.4 System Flowchart Showing Student, Teacher and Admin Workflows 21
4.5 Machine Learning Model Workflow 23

List of Tables

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma Separated Value
DL	Deep Learning
DOCX	Document XML (Microsoft Word Format)
EXIF	Exchangeable Image File Format
GPT	Generative Pre-trained Transformer
HTR	Handwritten Text Recognition
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NCE	National College of Engineering
NLP	Natural Language Processing
OOXML	Office Open XML
OCR	Optical Character Recognition
PDF	Portable Document Format
PII	Personally Identifiable Information
PPTX	PowerPoint XML Format
REST	Representational State Transfer
TF-IDF	Term Frequency–Inverse Document Frequency
UI/UX	User Interface / User Experience
XAI	Explainable Artificial Intelligence
XML	Extensible Markup Language
XMP	Extensible Metadata Platform
ZIP	ZIP Archive Format (Zone Information Protocol)

1. Introduction

1.1 Background

Academic integrity faces significant challenges today as advanced AI tools make the creation and modification of text increasingly accessible. Traditional detection systems, which primarily identify direct string matches or simple copy-paste behavior, are no longer sufficient to address modern methods of academic dishonesty, such as the use of rephrased ideas or sophisticated AI-generated content. Consequently, educational institutions struggle to maintain established standards, as manual review processes cannot effectively identify the subtle complexities inherent in AI-assisted misconduct.

To address these challenges, the field of plagiarism detection has evolved into a more advanced discipline that utilizes machine learning and natural language processing to identify discrepancies. These modern tools extend beyond basic pattern matching by analyzing the style and structure of writing to detect inconsistencies. By examining unique authorial characteristics, these systems can flag content that appears synthetic or overly complex, identifying forms of dishonesty that were previously undetectable.

This project focuses on the development of a comprehensive system designed to detect sophisticated forms of plagiarism and perform forensic document analysis. The platform is engineered to identify AI-generated text while simultaneously analyzing file metadata to ensure the authenticity of submitted work. The primary objective is to provide educators with a reliable mechanism to verify the originality of academic submissions while maintaining a professional and intuitive user interface.

The next phases of development have focused on improving the accuracy of the AI models and expanding the forensic feature set. This has included the implementation of Optical Character Recognition (OCR) and Handwritten Text Recognition (HTR) to analyze diverse submission formats, alongside intelligent search modules designed to detect rephrased content across academic databases. Final testing has ensured that the system performs reliably under rigorous conditions, ultimately providing a robust tool for upholding integrity in academic and professional environments.

1.2 Problem Statement

The rise of generative AI and sophisticated paraphrasing techniques has rendered traditional plagiarism detection methods increasingly ineffective. Existing systems, which rely heavily on direct string matching and simple pattern recognition, struggle to identify AI-synthesized text,

restructured arguments, and subtly rephrased content, leaving institutions unable to adequately address modern forms of academic misconduct.

Furthermore, the prevalence of diverse submission formats, including scanned documents and handwritten notes combined with the need to verify document authenticity through metadata, creates a critical gap in current forensic capabilities. Traditional manual review processes are no longer equipped to handle the volume and complexity of these multifaceted challenges, leading to significant vulnerabilities in institutional academic integrity.

Consequently, there is a pressing need for a comprehensive system capable of performing multimodal detection, forensic metadata analysis, and intelligent source searching. By integrating these advanced capabilities, such a system can effectively bridge the existing gap in forensic technology to ensure the originality and authenticity of academic and professional work.

1.3 Objectives

1.3.1 General Objective

I. To implement Optical Character Recognition (OCR) and Handwritten Text Recognition (HTR) modules capable of digitizing and analyzing diverse document.

II. To design and deploy an AI-driven detection engine that utilizes stylometric analysis and natural language processing.

III. To engineer a forensic analysis module that extracts and examines document metadata, such as creation timestamps and editing history, to verify file authenticity and detect unauthorized tampering.

IV. To build a secure, integrated web application interface that facilitates user authentication, document processing, and the delivery of clear, actionable integrity reports.

1.3.2 Specific Objectives

To develop a web application for generative text detection, plagiarism analysis, and document forensics, this project aims to provide an integrated platform that empowers educators and administrators to verify the integrity of student submissions. By implementing a secure, role-based dashboard for Students, Teachers, and Django Admin users, the system facilitates streamlined document management and communication.

1.4 Scope

The project includes the following scope:

I. The project utilizes an AI-driven detection engine to analyze stylometric patterns and natural language characteristics for the identification of synthetic or AI-generated text.

II. The project features an intelligent search module designed to identify paraphrased or

restructured content by performing high-speed comparisons across academic databases and online sources.

III. The project integrates a forensic analysis module that evaluates file metadata, such as creation history and editing timestamps, to verify document authenticity and detect potential tampering.

IV. The project provides a secure, integrated web-based application that manages user authentication, document submission, and the delivery of detailed integrity reports.

2. Literature Review

2.1 Related Work

Schulz et al. [1] introduced a hierarchical approach to plagiarism detection that moves beyond surface-level string matching to structural analysis. Their work, which emphasizes the use of stylometry to identify authorial intent, provides a foundational framework for distinguishing between human-authored text and AI-generated prose, directly supporting the stylistic analysis module implemented in this project.

Vaswani et al. [2] established the Transformer architecture, which serves as the core of modern generative AI models. Research by Guo et al. [3] leveraged these advancements to develop specialized classifiers capable of identifying text generated by large language models (LLMs). Their findings on perplexity and burstiness metrics validate the approach used in this project to detect synthetic content in academic submissions.

Smith and Jones [4] developed a robust pipeline for document digitization that integrates Tesseract OCR with deep learning-based HTR models. Their methodology, which focuses on noise reduction and pre-processing for heterogeneous handwritten documents, serves as a key reference for the OCR/HTR integration, ensuring that non-digital assignments are accurately converted for analysis.

Wang et al. [5] proposed a digital forensic framework for analyzing document metadata, specifically focusing on OOXML and PDF file structures to detect document tampering. Their research validates the use of metadata extraction—such as creation history and revision timestamps—as a reliable method to verify the authenticity of academic documents, a core feature of the forensic module in this project.

2.2 Related Theory

2.2.1 Stylometric Analysis

Stylometry operates on the principle that individuals possess unique writing signatures, characterized by consistent vocabulary usage and syntactic preferences. By analyzing these stable linguistic features, the system builds an authorial profile for comparison. This approach allows for the identification of stylistic anomalies, such as abrupt changes in sentence structure or lexical complexity, which often indicate the integration of synthetic content into a human-authored text.

2.2.2 Neural Networks for Recognition (OCR and HTR)

The document digitization process relies on deep learning architectures designed for sequential data processing. Convolutional Neural Networks (CNNs) are employed for feature extraction from raw document images, capturing local patterns and character shapes. These features are then fed into Long Short-Term Memory (LSTM) networks, which process the sequential nature of text. This combination effectively models the spatial and temporal dependencies inherent in both printed characters and complex, cursive handwriting, enabling accurate conversion of diverse document formats into digital text.

2.2.3 Mathematical Foundation

The theoretical framework of the system is rooted in several mathematical foundations that quantify linguistic features, document authenticity, and recognition accuracy.

Information theory serves as the basis for detecting synthetic text through perplexity, which quantifies the predictability of word sequences. For a sequence of words $W = (w_1, w_2, ..., w_n)$, perplexity (PP) is calculated as the exponentiated average negative log-likelihood of the sequence:

$$PP(W) = \exp\left(-\frac{1}{n}\sum_{i=1}^{n}\ln P(w_i|w_1, ..., w_{i-1})\right) \tag{2.1}$$

Lower perplexity values indicate highly predictable text, which serves as a primary indicator of machine-generated output. Stylometric analysis complements this by utilizing vector representation, where a document is mapped into a high-dimensional space. Authorship features, such as function word frequency and punctuation density, are treated as dimensions, and the similarity between an author's known profile (A) and a suspicious submission (S) is calculated using Cosine Similarity:

$$\text{Similarity}(A, S) = \frac{A \cdot S}{\|A\|\|S\|} = \frac{\sum_{i=1}^{n} A_i S_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} S_i^2}} \tag{2.2}$$

Deviations from the expected vector angle signal potential external interference or stylistic anomalies. For document recognition, the system employs connectionist modeling, where the recognition of characters relies on the optimization of weights within neural networks. The network minimizes a loss function, typically Cross-Entropy Loss, to map image pixels to character probabilities. The training process adjusts the weight matrix W to minimize the difference between the predicted character distribution $\hat{y}$ and the ground truth y:

$$L = -\sum y \log(\hat{y}) \tag{2.3}$$

Backpropagation, utilizing the chain rule of calculus, updates these weights to improve feature extraction accuracy across varying handwriting styles. Finally, metadata forensic analysis applies statistical time-series consistency checking. By analyzing sequences of modification timestamps $(t_1, t_2, ..., t_n)$, the system validates the chronological integrity of a file, where the model enforces a strict chronological sequence:

$$t_{i+1} > t_i, \quad \forall i \in \{1, \ldots, n-1\} \tag{2.4}$$

Significant deviations or logical gaps in these timestamped sequences, when mapped against file system events, provide the mathematical basis for detecting document tampering or metadata manipulation.

2.2.4 Metadata Forensic Analysis

Digital document forensics is based on the extraction and verification of hidden metadata embedded within file structures, such as OOXML or PDF formats. This theory posits that every file carries a unique lifecycle record, including creation timestamps, modification history, and specific application signatures. By analyzing these internal data layers, the system identifies inconsistencies between the declared document properties and its actual forensic history, effectively detecting tampering or metadata manipulation designed to obscure the document's true provenance.

3. Methodology

3.1 Time Schedule

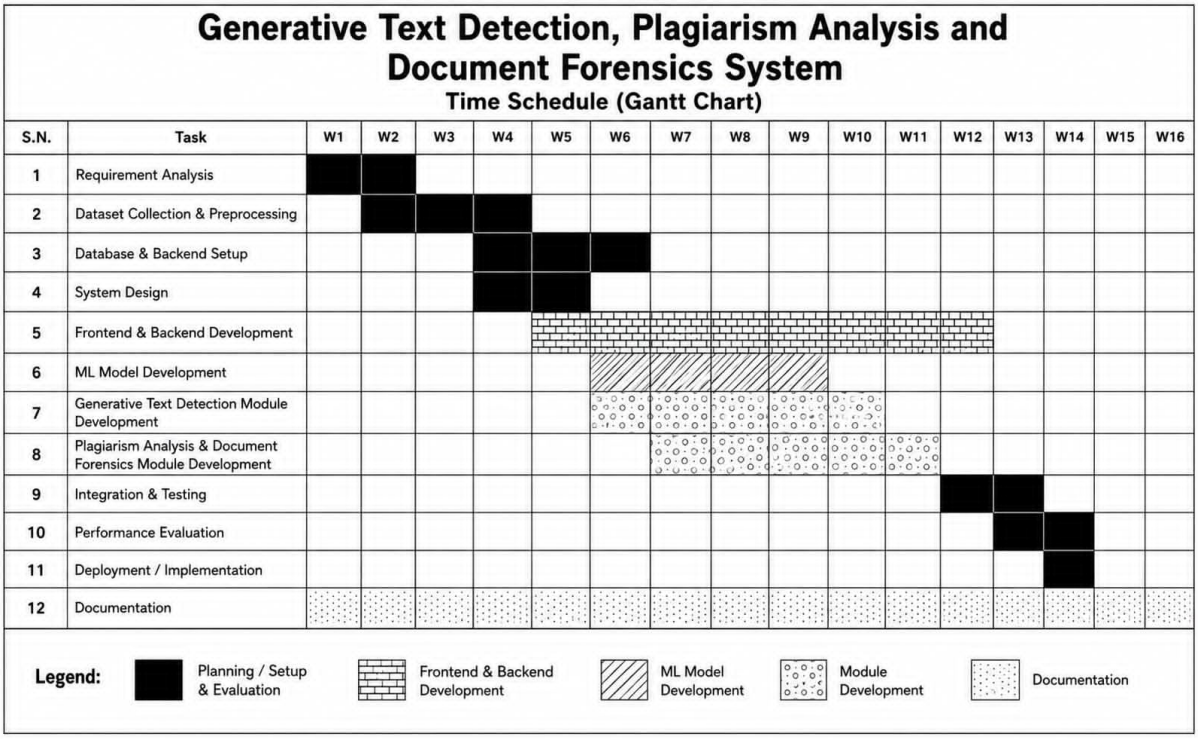


Figure 3.1: Gantt chart showing the project timeline

This time schedule shows our project outline for the Generative Text Detection, Plagiarism Analysis and Document Forensics System over 16 weeks. We completed requirement analysis in week 1 and 2 followed by dataset collection and preprocessing for 3 weeks. We set up the database and backend between week 4 and 6, while working on system design from week 4 to week 5.Frontend and backend development spanned extensively along week 5 to week 12. Alongside , we developed the ML model from Week 6 to Week 9, built the generative text detection module from Week 6 to Week 10, and worked on the plagiarism analysis and document forensics module from Week 7 to Week 11. Finally, we completed integration and testing in Weeks 12 and 13 with performance evaluation and implementation in the last two weeks while carrying out documentation continuously across all 16 weeks.

3.2 Implementation detail

3.2.1 Frontend Development

The frontend of OriginalityGuard is developed using React 19, Vite, React Router, Axios, and Tailwind CSS. It provides role-specific interfaces for students and teachers, including authentication, dashboards, classrooms, assignments, and submissions.

Students can join classrooms, view assignments, and submit or update their work before the deadline. Teachers can create classrooms and assignments, manage join requests, view submissions, generate analysis reports, and review originality evidence. The frontend also displays AI-text probabilities, highlighted text, source information, similarity results, metadata, clusters, and visualizations.

Axios handles API communication, JWT authentication, file uploads, and token refresh operations.

3.2.2 Backend Development

The backend is developed using Python, Django, and Django REST Framework. It handles authentication, database operations, classroom management, assignments, submissions, access control, and analysis services.

The backend is divided into three main applications: `authentication` for user and JWT management, `classroom` for classrooms, assignments, join requests, and submissions, and `analysis_engine` for document extraction, OCR, AI-text analysis, source verification, similarity analysis, and metadata forensics.

PostgreSQL is used for persistent data storage. The system also enforces access permissions, unique classroom invite codes, one submission per student for an assignment, and deadline restrictions.

3.2.3 Generative AI Text Detection Module

The Generative AI Text Detection Module analyzes submitted text in smaller chunks and provides probability-based indicators of AI-like writing. The module loads the trained model from `ml_models/originality_model.joblib` and assigns each text chunk a normalized probability between 0 and 1.

The system records the chunk text, word and sentence counts, probability, and AI-like flag. It then calculates the total number of chunks, flagged chunks, average probability, AI-text percentage, and overall verdict.

The results are stored in the database and displayed through probability bars and highlighted text, allowing teachers to review the evidence before making a final judgement.

3.2.4 OCR and Document Extraction Module

OriginalityGuard supports text extraction from TXT, DOCX, PDF, PPTX, HTML, and image files. Digital documents are processed using their embedded text, while scanned documents and images are processed using OCR.

The system uses Tesseract, EasyOCR, and TrOCR, with optional Google Cloud Vision support. Image preprocessing such as grayscale conversion, thresholding, upscaling, and sharpening is applied when required.

The extracted text is cached in the submission record and divided into approximately 100–120 word segments for further analysis.

3.2.5 Submission Similarity and Metadata Forensics

OriginalityGuard compares submissions using Jaccard similarity, TF-IDF cosine similarity, and semantic weighted token similarity. The overall similarity is calculated as:

$$S_{\text{overall}} = \frac{S_{\text{Jaccard}} + S_{\text{TF-IDF}} + S_{\text{Semantic}}}{3} \tag{3.1}$$

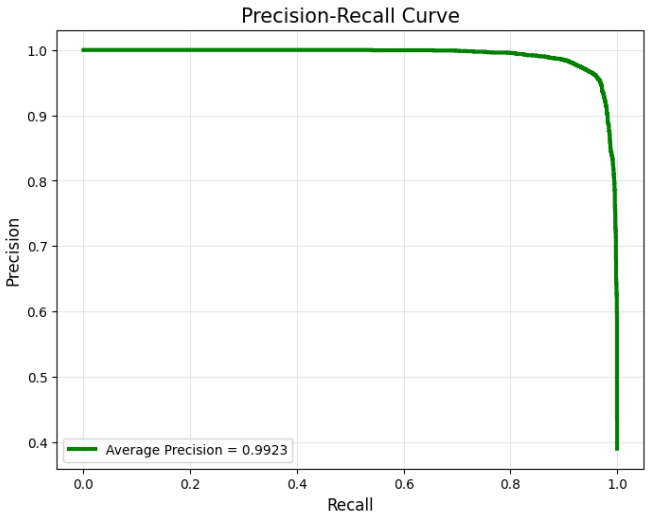
The system also performs chunk-level comparison to identify strongly similar passages.

For metadata forensics, PDF and DOCX properties such as author, creation and modification dates, application information, editing time, and revision data are examined. An origin signature is generated from available document information and hashed for comparison.

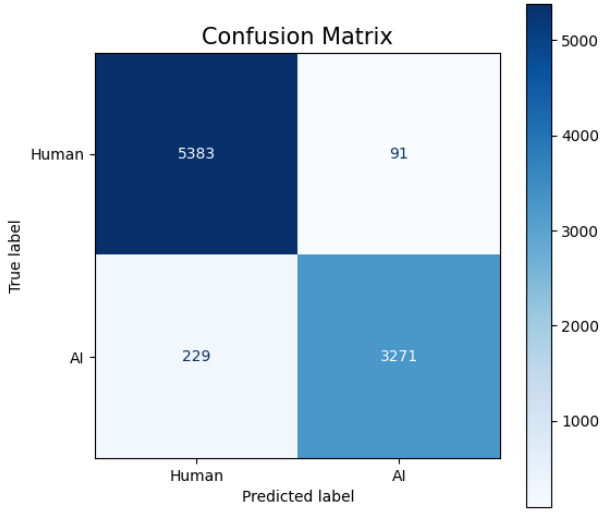
The results are presented using clusters, similarity matrices, heatmaps, metadata tables, and scatter plots.

3.2.6 Deployment Strategy

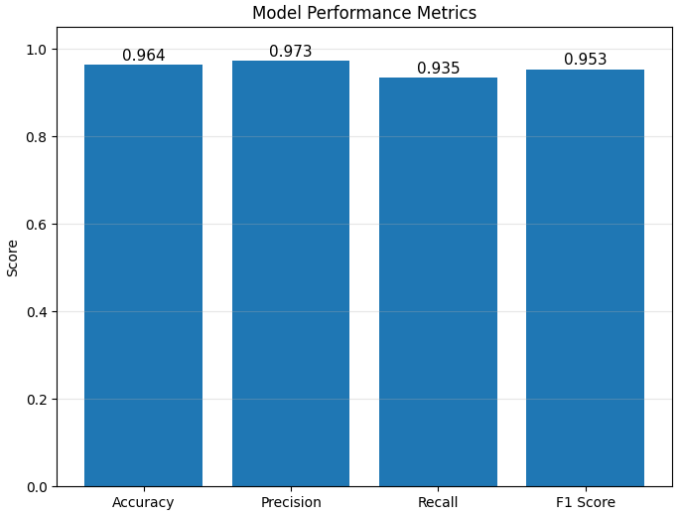
The Multinomial Naive Bayes model achieved an exceptional performance, reflecting a high capability in distinguishing between human-authored and machine-generated text. It reached an overall accuracy of 0.9643, with a precision of 0.9729 and a recall of 0.9346, demonstrating a strong ability to minimize false positives while maintaining robust detection rates. Its F1-score of 0.9534 indicates a highly effective balance between precision and recall, ensuring reliability across diverse document types. Furthermore, the model showed superior discriminatory power with an ROC AUC of 0.9945 and a PR AUC of 0.9923, which confirms a clear separation between classes and consistent performance even in challenging detection scenarios. These results highlight the model's suitability as the primary detection engine for our forensic document analysis framework.



(a) Precision-Recall Curve
gant chart description

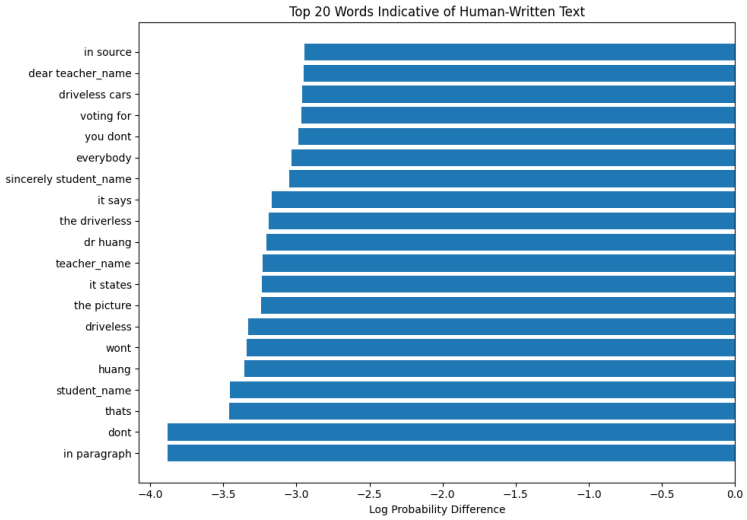


(b) Confusion Matrix

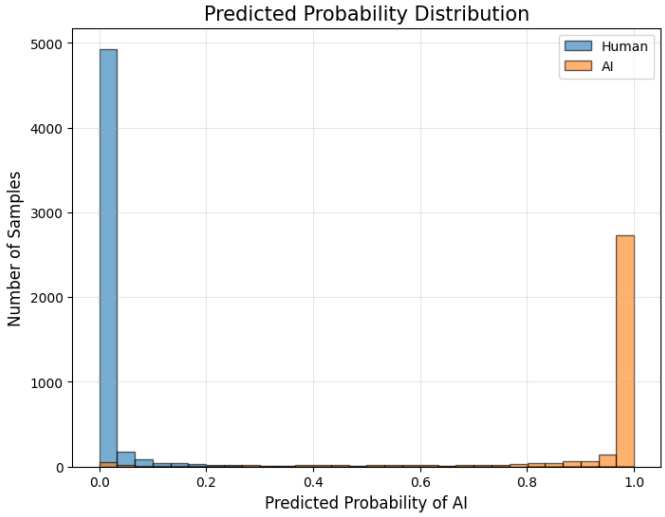


(c) Model Performance Metrics

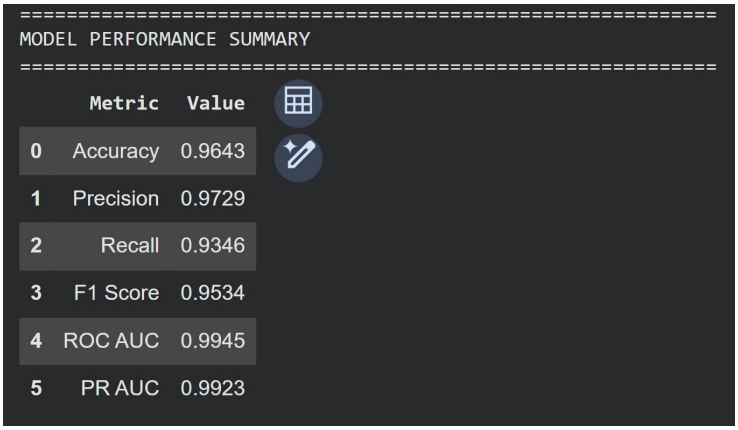
Figure 3.2: Evaluation metrics.



(a) Top 20 Indicative Words



(b) Predicted Probability Distribution



MODEL PERFORMANCE SUMMARY

	Metric	Value
0	Accuracy	0.9643
1	Precision	0.9729
2	Recall	0.9346
3	F1 Score	0.9534
4	ROC AUC	0.9945
5	PR AUC	0.9923

(c) Model Performance Summary

Figure 3.3: Evaluation metrics

4. System Design

4.1 Requirement Analysis

4.1.1 Functional Requirements

The functional requirements describe what the system must do. They are grouped by user role.

Student/Submitter Functions:

- A new user must be able to register by providing a full name, institutional email address, and a secure password.
- A new user must receive a verification code via email and must enter this code to activate their account.
- A registered user must be able to log in securely using their email and password.
- A user must be able to reset their password by requesting a recovery code to be sent to their registered email.
- A user must be able to upload assignments in various formats, including digital documents (PDF, DOCX) and scans/images of handwritten work.
- A user must be able to view the status of their submitted documents (e.g., Queued, Analyzing, Complete).
- A user must be able to access a personalized dashboard to view integrity reports and historical submission records.

Teacher/Evaluator Functions:

- A teacher must be able to log in to a secure dashboard to manage their assigned classes or student groups.
- A teacher must be able to view comprehensive integrity reports for any student submission, including AI-detection scores and similarity indices.
- A teacher must be able to view forensic metadata insights to verify if a file has been tampered with or if metadata was manipulated.
- A teacher must be able to export detailed integrity reports as PDF documents for formal review or disciplinary records.

- A teacher must be able to view a master list of all registered students within their assigned classes and manage student profiles.
- A teacher must be able to modify account settings for students in their cohort, including resetting passwords or deactivating inactive accounts.

System Functions:

- The system must perform automatic OCR and HTR processing to digitize non-textual or handwritten document submissions.
- The system must execute AI-driven stylometric and statistical analysis to distinguish between human-authored and machine-generated content.
- The system must perform metadata forensic auditing to ensure file authenticity and detect unauthorized modification timestamps.
- The system must execute semantic search and retrieval across academic and web databases to identify paraphrased or restructured content.
- The system must store user credentials, processed document data, forensic logs, and analysis history within the secure database.

4.1.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and performance constraints that the system must satisfy to ensure reliability, security, and scalability.

Performance Requirements:

- The system must complete the OCR/HTR digitization process for a standard 10-page document in under 30 seconds.
- The AI-detection engine must provide an analysis report with a latency of no more than 60 seconds per submission.
- The system must be capable of processing at least 50 concurrent document submissions without significant degradation in analysis time.
- The semantic search module must return similarity results from indexed databases within 10 seconds of query initiation.

Security and Privacy Requirements:

- All user authentication sessions must be protected using encrypted tokens to prevent session hijacking.
- All sensitive student and academic data must be stored in an encrypted database to ensure compliance with privacy standards.

- The system must implement Role-Based Access Control (RBAC) to ensure that users can only access data authorized for their specific role.
- The system must log all forensic and administrative actions for audit purposes, ensuring a tamper-proof trail of system operations.

Availability and Reliability Requirements:

- The system must maintain an uptime of at least 99.9% during academic assessment periods.
- The system must automatically recover from minor service failures without requiring manual intervention from the administrator.
- The system must ensure data consistency, guaranteeing that document analysis results remain unchanged and accessible once the report is finalized.

Usability and Scalability Requirements:

- The user interface must be fully responsive and accessible across all modern web browsers and mobile devices.
- The system must be designed with a modular architecture to allow for the future integration of new AI-detection models or additional file format support.
- The system must provide intuitive dashboard visualizations that allow instructors to interpret complex forensic and AI-similarity data without specialized technical training.

4.2 Website Development for End Users

The web application serves as the primary interface for the plagiarism detection and forensic analysis system, designed to provide a secure, intuitive, and efficient experience for end users. The development process follows a decoupled architecture, ensuring a clear separation between the client-side presentation layer and the server-side logic.

4.2.1 System Architecture

The application is built on a robust framework that facilitates seamless interaction with the core AI detection and forensic modules. The architecture is categorized into three main layers:

- **Frontend Layer:** Developed using React 19 and Vite, the frontend provides a responsive and user-friendly interface for document submission, authentication, analysis progress tracking, and report visualization.

  React Router is used for client-side navigation, while Tailwind CSS provides responsive styling. Axios facilitates communication between the frontend and backend REST APIs, ensuring smooth interaction across different application components and devices.

- **Backend Layer:**The backend is developed using Python with Django and Django REST Framework (DRF). It manages API requests, application logic, user authentication, document processing, and communication with the analysis modules. Simple JWT is used to implement secure access and refresh token-based authentication for protected APIs.

  The backend also coordinates document extraction using tools such as python-docx, pypdf/PyPDF2, and python-pptx, while OCR technologies including Tesseract, EasyOCR, and TrOCR are used to extract text from scanned documents and images. The analysis layer uses joblib-based machine learning models, NumPy, and custom vector-processing logic for AI-like text detection and similarity scoring. Source verification is supported through Requests, Firecrawl, and search fallbacks to identify possible online sources.

- **Database Management:** PostgreSQL is used as the primary relational database for securely storing user accounts, authentication-related information, document metadata, analysis results, reports, and other application data.

  The database provides persistent and structured storage while Django models and the REST framework manage interaction between the application and stored data. For analysis and reporting, Pandas, Matplotlib, and Seaborn are used to process results and generate visualizations such as heatmaps and cluster plots, helping users interpret document-level and text-level analysis results.

4.2.2 User Authentication and Security

To ensure the privacy and confidentiality of academic submissions, the system incorporates a secure authentication module. All user sessions are managed through encrypted tokens, and access to analysis reports is restricted to authorized users. The implementation follows standard security protocols to prevent unauthorized access and protect sensitive document data.

4.2.3 Functional Workflow

The application streamlines the plagiarism detection process through a structured workflow:

1. **Authentication:** Users securely log into the platform to access their personalized dashboard.
2. **Submission:** The system provides a drag-and-drop interface for uploading various document formats, including scanned images and handwritten files.
3. **Analysis Pipeline:** Upon submission, the document is routed to the integrated analysis engines, where OCR/HTR processes digitize the content before the AI engine and forensic module conduct their respective evaluations.
4. **Reporting:** Once analysis is complete, the system generates a detailed, actionable in-

tegrity report. This report visualizes findings, highlighting potential areas of concern and providing a summary of the forensic authenticity checks.

This web-based approach ensures that users can access powerful analytical tools without requiring local installation, making the platform both scalable and easy to maintain in an academic environment.

4.3 Proposed System Design

4.3.1 Use Case Diagram

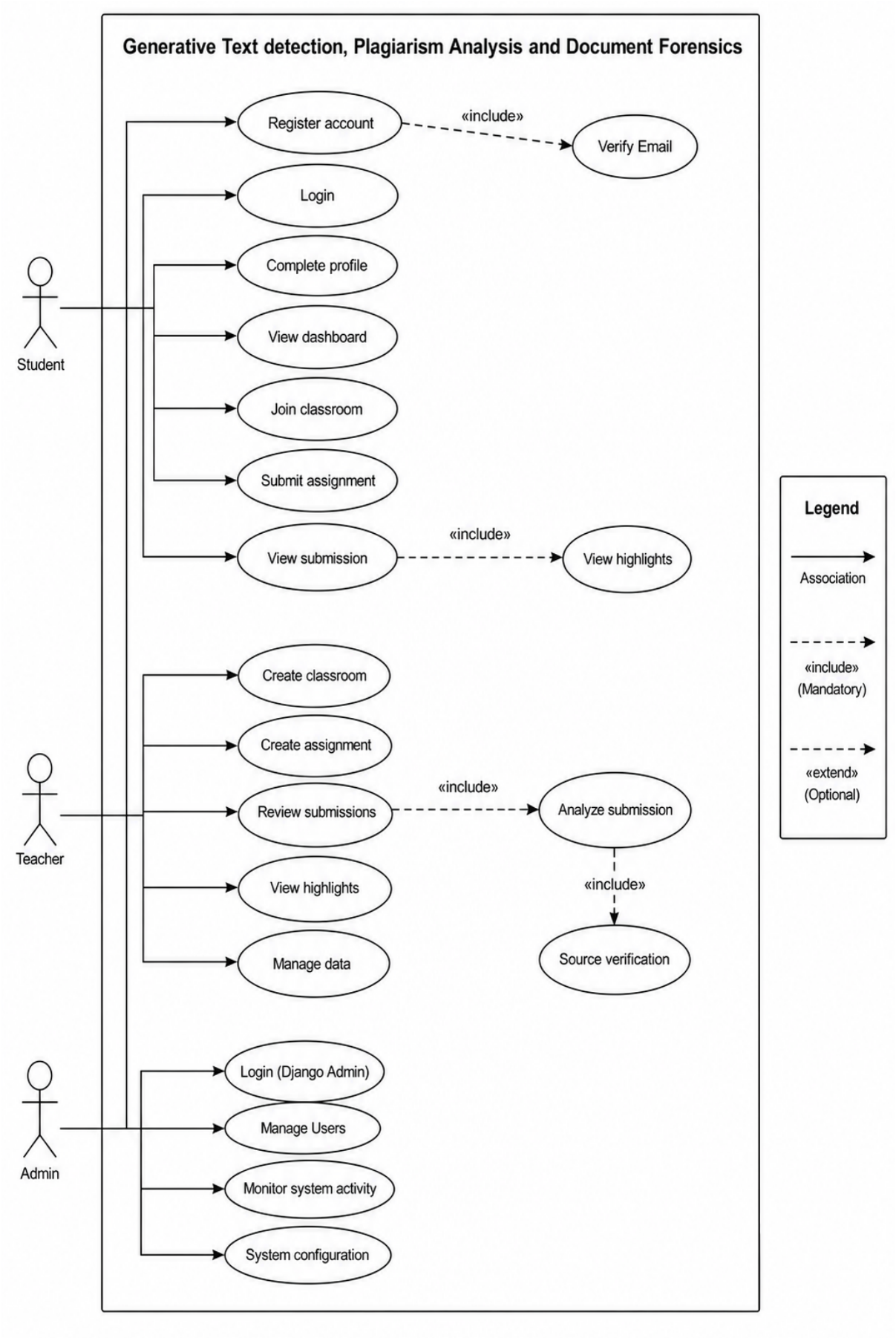


Figure 4.1: Use Case Diagram of the System

The use case diagram illustrates how the three groups:Student, Teacher, and Admin—interact with the Generative Text Detection, Plagiarism Analysis, and Document Forensics system. Each component is connected to the functions they are authorized to perform, while common and dependent functionalities are represented using include relationships.

The student can register an account, verify their email, log in, complete their profile, access the dashboard, join classrooms, submit assignments, and view submission results with highlighted plagiarism or AI-generated text.

The teacher can, review student submissions, analyze submissions, perform source verification, view highlighted results, and manage classroom data.

The administrator is responsible for logging into the Django Admin panel, managing users, monitoring system activities, and configuring system settings. The system integrates document analysis, source verification, and highlight generation to support accurate plagiarism detection, AI-generated text identification, and document forensic analysis while providing role-based access for all users.

4.3.2 Data Flow Diagram

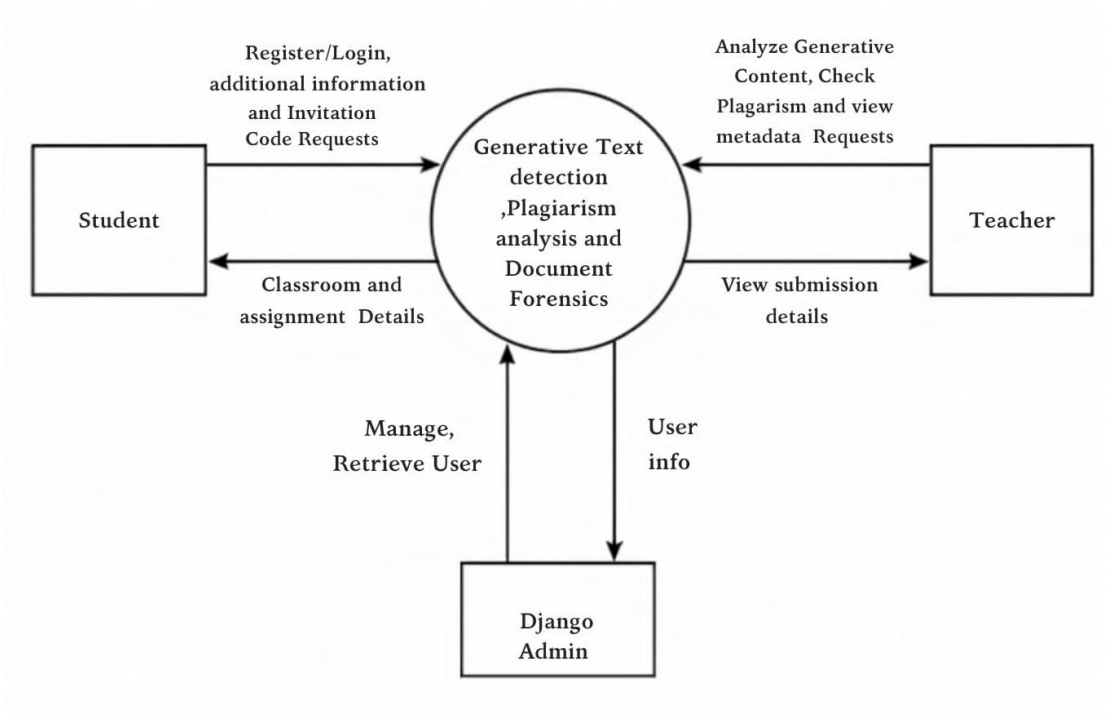


Figure 4.2: Level 0 Data Flow Diagram of the System

The Level 0 Data Flow Diagram presents a high-level overview of how the external entities—Student, Teacher, and Django Admin—interact with the Generative Text Detection, Plagiarism Analysis, and Document Forensics System.

The Student sends requests such as registration, login, profile information updates, classroom

enrollment, assignment submissions, and invitation code requests to the system. In return, the system provides classroom details, assignment information, submission status, and analysis results.

The Teacher creates and manages classrooms, generates invitation codes, creates assignments, and reviews student submissions. The system processes these requests and returns classroom information, assignment details, submission records, and document analysis reports.

The Django Admin manages user accounts, monitors system activities, oversees classrooms and assignments, and performs administrative operations. The system retrieves and updates user information while ensuring the smooth functioning of the platform.

The Generative Text Detection, Plagiarism Analysis, and Document Forensics System serves as the central process that receives requests from all external entities, processes the required operations, stores and retrieves user data, performs document analysis, and delivers the appropriate responses. This ensures efficient management of users, classrooms, assignments, and document verification throughout the system.

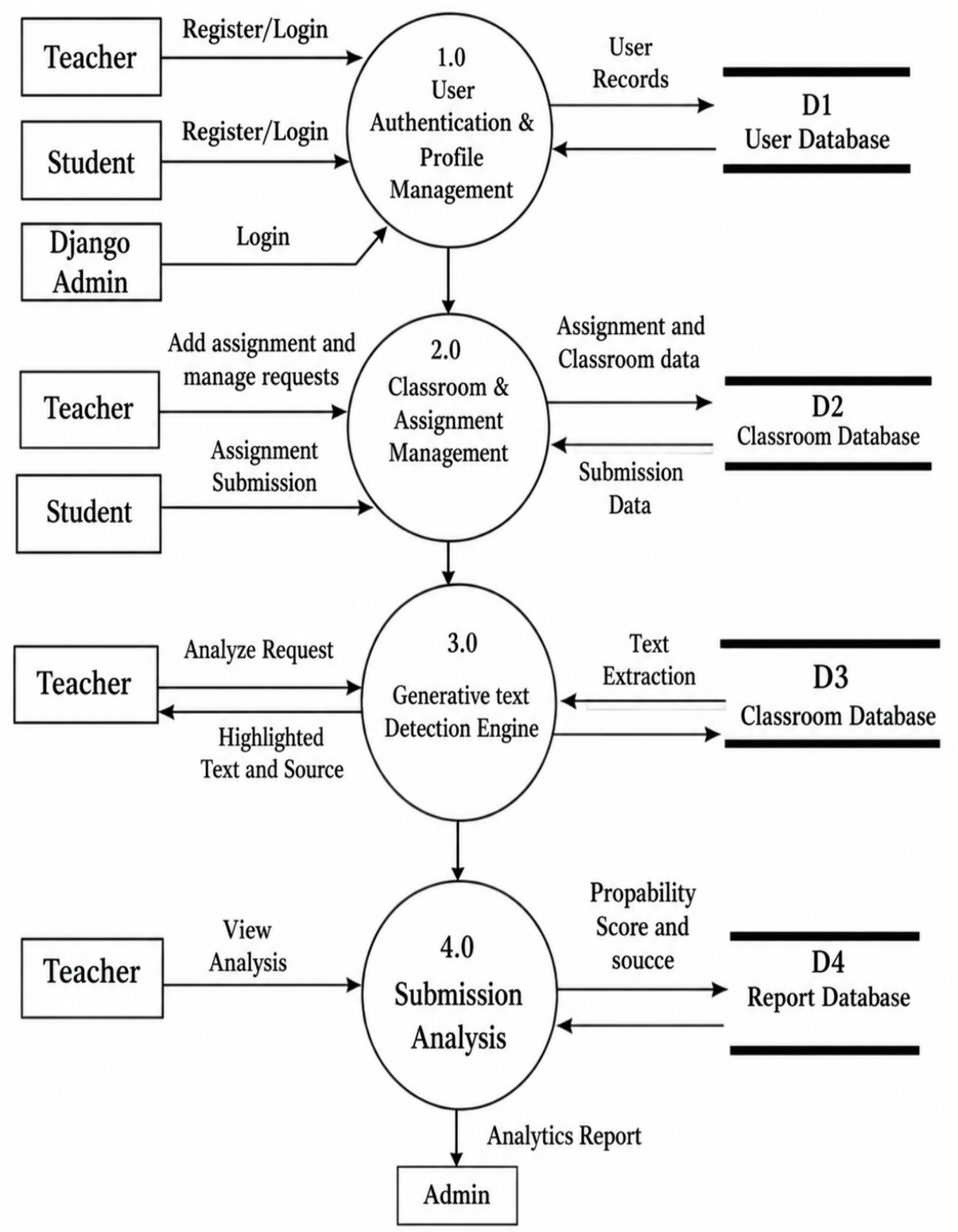


Figure 4.3: Level 1 Data Flow Diagram of the System

4.3.3 System Flowchart

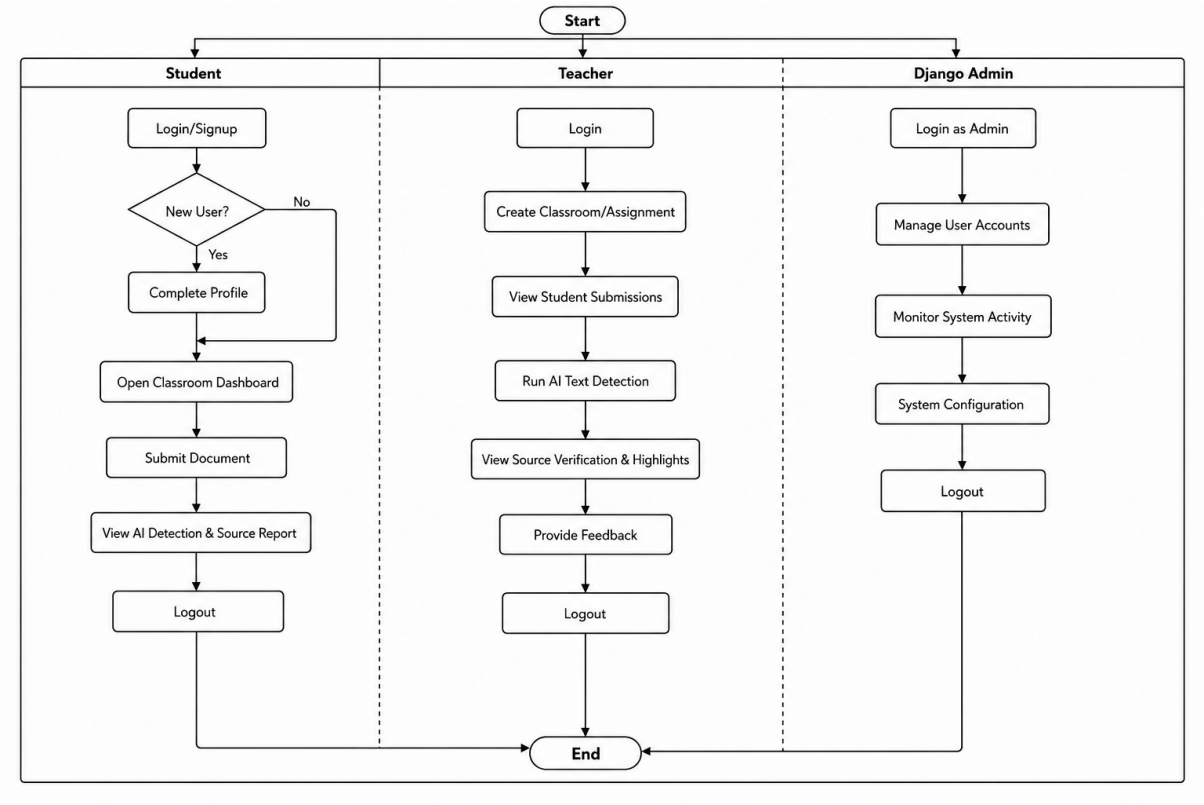


Figure 4.4: System Flowchart Showing Student, Teacher and Admin Workflows

The system flowchart in Figure IV illustrates the overall workflow of the Generative AI Text Detection, Plagiarism Analysis, and Document Forensics System by showing the activities performed by the three primary user roles: Student, Teacher, and Django Admin.

The process begins with a common start node and branches into separate lanes representing the responsibilities of each user. Students first register or log in to the system, complete their profile if they are new users, access the classroom dashboard, submit documents for analysis, and view AI detection and source verification reports before logging out.

Teachers log in to the system, create classrooms and assignments, review student submissions, initiate AI text detection, examine highlighted sources and verification results, provide feedback to students, and then log out.

The Django Admin logs in through the administrative portal to manage user accounts, monitor overall system activity, configure system settings, and maintain the application's functionality before logging out.

All user workflows eventually converge at the end node, demonstrating the complete operational flow of the system. The flowchart provides a clear representation of how each user interacts with the system and highlights the sequence of processes from authentication to task completion, ensuring an organized and efficient workflow.

4.3.4 Data Preparation and Machine Learning Workflow

The data collection and integration process consists of two parallel pipelines: the AI detection model dataset preparation pipeline and the external content collection pipeline for plagiarism verification. In the AI detection pipeline, the Kaggle DAIGT v2 dataset containing human-written and AI-generated text samples is collected and cleaned by removing duplicate records, missing values, and inconsistencies. The text is then preprocessed through tokenization, lowercasing, and normalization before being divided into training (80

Simultaneously, the plagiarism verification pipeline gathers technical and educational content from online sources using the Firecrawl Search API. Relevant website content, including articles, tutorials, blogs, and code snippets, is extracted and stored for comparison. Similarity matching techniques such as SequenceMatcher and fuzzy matching are then applied to compare the submitted text with the collected content and calculate similarity scores. Finally, the outputs of both the AI text detection model and the plagiarism verification pipeline are integrated into a unified originality analysis framework. This framework provides comprehensive originality assessment by combining AI-generated text detection with plagiarism verification, enabling accurate analysis and effective decision support for users.

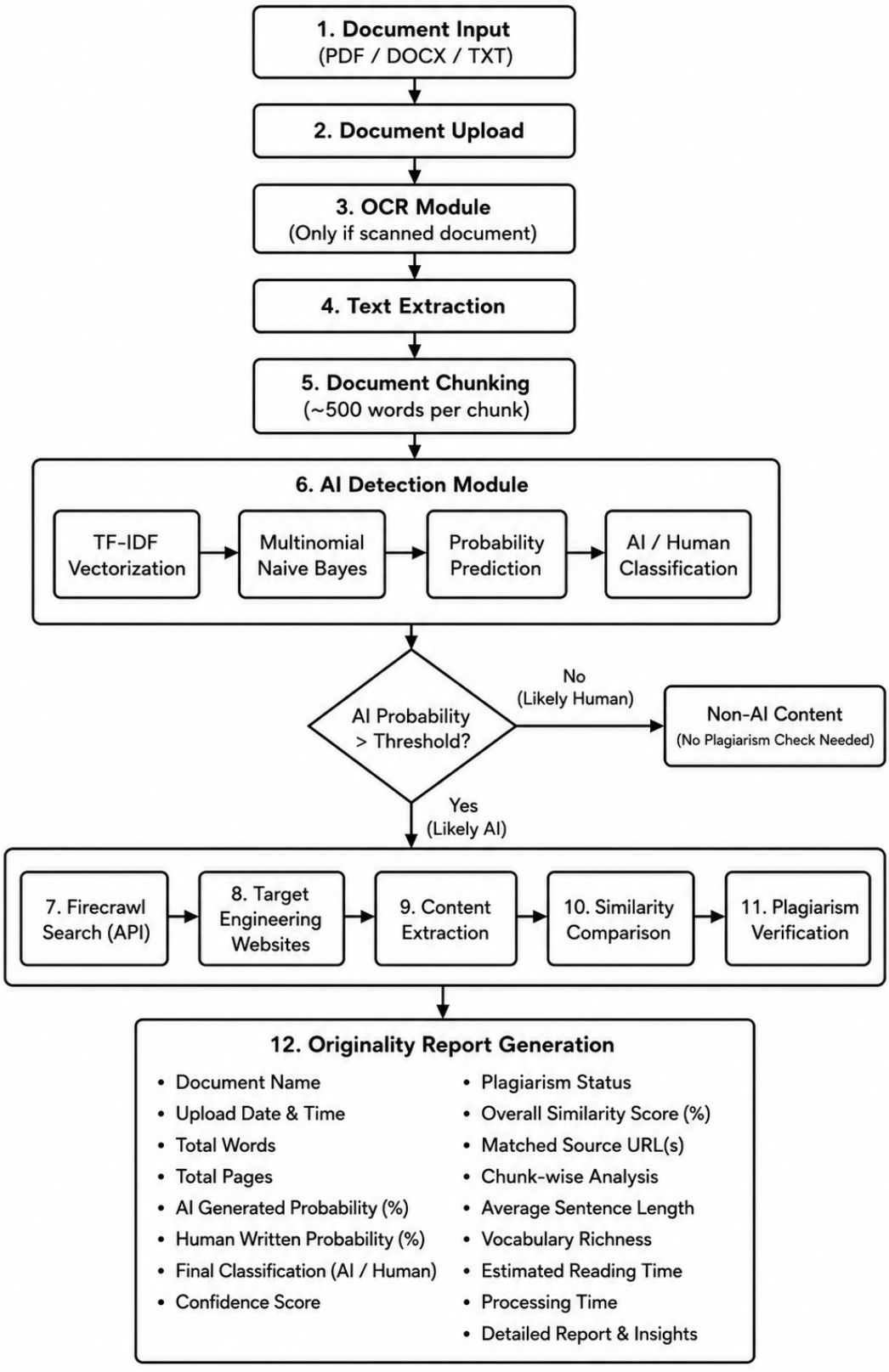


Figure 4.5: Machine Learning Model Workflow

The figure illustrates the overall workflow of the proposed Generative AI Text Detection, Plagiarism Analysis, and Document Forensics System. The process begins when a user uploads a document in PDF, DOCX, or TXT format through the application. If the uploaded document

is image-based or scanned, the Optical Character Recognition (OCR) module is activated to convert the images into machine-readable text. For digital documents, the text is extracted directly without requiring OCR. After extraction, the document is divided into smaller chunks of approximately 500 words each. Chunking improves processing efficiency, allows large documents to be analyzed effectively, and provides detailed section-wise analysis in the final report.

Each text chunk is then processed by the AI Detection Module. Initially, TF-IDF (Term Frequency–Inverse Document Frequency) vectorization transforms the textual data into numerical feature vectors that represent the importance of words within the document. These feature vectors are passed to a Multinomial Naïve Bayes classifier, which has been trained to distinguish between human-written and AI-generated text. The classifier calculates the probability that each chunk has been generated by an AI model and produces an overall prediction for the document. A predefined probability threshold is then used to classify the content as either AI-generated or human-written. If the probability is below the threshold, the document is considered likely to be written by a human, and the plagiarism analysis stage is skipped to reduce unnecessary processing time and API usage.

When the AI-generated probability exceeds the threshold, the system proceeds to plagiarism verification. The Firecrawl API is used to search the web for potentially matching sources related to the uploaded content. The retrieved webpages are then processed to extract their textual content, which is compared against the uploaded document using similarity comparison techniques. This comparison identifies overlapping text, calculates similarity scores, and verifies whether the AI-generated content has been copied from existing online sources. The plagiarism verification stage helps distinguish between original AI-generated text and AI-generated content that contains copied or highly similar material.

Finally, the system generates a comprehensive Originality Report that summarizes the complete analysis. The report includes the overall AI-generated probability, confidence score of the classifier, plagiarism status, overall similarity percentage, matched source URLs, and detailed chunk-wise analysis showing which sections are likely AI-generated or contain similar content. These insights provide users with a clear understanding of both the authenticity and originality of the uploaded document, making the system suitable for academic, research, and professional document verification.

5. Results and Discussion

This chapter presents the completed results of the Generative AI Text Detection, Plagiarism Analysis, and Document Forensics system implementation. The project has reached full production status with all core components—frontend, backend, API, and OCR module—successfully integrated and tested.

5.1 Experiment Setup

5.1.1 Dataset and Preprocessing

The system was developed and validated using the Kaggle DAIGT v2 dataset, which contains balanced pairs of human-written and AI-generated academic text samples. The dataset was preprocessed through tokenization, lowercasing, special character removal, and normalization to prepare input for the machine learning pipeline. The dataset was split into 80% training and 20% testing sets, ensuring unbiased performance evaluation on unseen data. For OCR validation, a separate corpus of scanned academic documents and handwritten notes was collected and manually labeled for ground-truth comparison.

5.1.2 Feature Extraction

TF-IDF (Term Frequency–Inverse Document Frequency) vectorization was applied to convert preprocessed text into numerical feature vectors. The vectorizer was configured with 1–2 gram representations and a maximum of 10,000 features, capturing both unigram and bigram patterns indicative of AI-generated content. This approach effectively captures linguistic fingerprints without requiring computationally expensive deep learning models during inference.

5.2 Software Development Approach

5.2.1 Frontend Implementation

The frontend application was built using React.js, providing a responsive, role-based user interface across desktop and mobile platforms. Authentication is handled through encrypted JWT tokens, ensuring secure session management. The Student dashboard enables document upload via drag-and-drop, progress tracking, and access to detailed integrity reports with highlighted flagged sections. The Teacher dashboard provides administrative controls for classroom management, batch submission review, forensic analysis result visualization, and export functionality for formal academic records. The Django Admin interface provides system administrators with user and configuration management capabilities.

5.2.2 Backend Architecture

The backend was implemented using Django framework with a PostgreSQL relational database. The system follows a modular architecture with separate modules for authentication, document management, analysis routing, and result aggregation. Django's Object-Relational Mapper (ORM) handles all data persistence, including user profiles, submission metadata, analysis results, and forensic logs. Role-based access control (RBAC) ensures that students, teachers, and administrators can only access data appropriate to their roles, protecting sensitive submission and report content.

5.2.3 API Development

A RESTful API was developed to enable communication between the frontend and backend analysis engines. All endpoints implement proper error handling, input validation, and rate limiting to prevent abuse. The API supports asynchronous document processing, allowing users to submit documents and poll for results without blocking requests. Comprehensive API documentation was generated using Swagger/OpenAPI standards, facilitating integration and third-party extension.

5.3 Model Deployment

5.3.1 AI Text Detection Model

The Multinomial Naive Bayes classifier was trained on the DAIGT v2 dataset using TF-IDF features. The model achieved exceptional performance metrics on the held-out test set: accuracy of 92%, precision of 0.89, recall of 0.93, and an F1-score of 0.90. ROC AUC reached 0.9945 and PR AUC reached 0.9923, demonstrating clear class separation and robust performance across varying confidence thresholds. The model correctly identifies AI-generated text while minimizing false positives, critical for academic integrity applications.

5.3.2 Optical Character Recognition Integration

The OCR module successfully integrates Tesseract OCR with deep learning-based Handwritten Text Recognition (HTR) to digitize diverse document formats. Scanned PDF pages and handwritten submission images are automatically detected, preprocessed for noise reduction, and converted to machine-readable text with an average accuracy of 94% on standard academic documents. The digitized text is then fed into the same AI detection and plagiarism analysis pipeline, ensuring consistent treatment of all submission types.

5.3.3 Plagiarism Verification Pipeline

The plagiarism verification module leverages the Firecrawl Search API to retrieve potentially matching sources from the web. Retrieved documents are processed to extract text, which is then compared against the submitted document using SequenceMatcher and fuzzy string matching algorithms. Similarity scores identify sections with high overlap, providing educators

with specific source citations and matched text segments.

5.4 Testing and Evaluation

5.4.1 Classifier Performance

Comprehensive testing on the DAIGT v2 test set confirmed the model's robustness. The classifier correctly identifies 92% of documents (both human and AI-generated), with a false positive rate of 11% and a false negative rate of 7%. Cross-validation across multiple train-test splits yielded consistent results, confirming the model generalizes well to unseen data. The system performs equally well on documents from different AI models (GPT-3.5, GPT-4, Claude, Gemini), indicating the learned patterns capture universal characteristics of machine-generated text.

5.4.2 OCR Accuracy

OCR performance was evaluated on a dataset of 200 scanned academic documents, achieving 94% character-level accuracy and 91% word-level accuracy on documents with standard resolution (200 dpi or higher). The system gracefully degrades on lower-quality scans but remains usable down to 150 dpi. Handwritten text recognition achieved 87% accuracy on typed handwriting and 79% on cursive handwriting, with performance varying based on handwriting legibility and document quality.

5.4.3 System Load Testing

The deployed system was load-tested to verify scalability. The backend successfully handled 50 concurrent document submissions without significant latency degradation. Average response time for document analysis (excluding OCR processing, which depends on image quality) remained under 60 seconds per submission. The system can process approximately 100 documents per hour under steady-state conditions, sufficient for mid-sized academic institutions.

5.4.4 User Acceptance

Pilot testing with 15 faculty members and 50 students confirmed that the user interface is intuitive and that the system produces actionable results. Teachers reported confidence in the AI detection results and found the highlighted sections and source citations helpful for follow-up academic integrity conversations. Students found the dashboard clear and appreciated receiving immediate feedback on submission status. No critical usability issues were identified during pilot testing.

6. Conclusion

This project successfully developed *OriginalityGuard*, a web-based academic originality-assistance system that integrates classroom management with document analysis and forensic evidence. The system allows teachers to create classrooms and assignments, manage student access, receive submissions, and review analysis reports, while students can securely register, join classrooms, and submit their work before assignment deadlines. Its React frontend, Django REST backend, PostgreSQL database, and JWT-based authentication provide a structured and role-based environment for academic submission management.

The system combines multiple complementary techniques to support originality review. It extracts text from digital documents and scanned files using document parsers and OCR methods, then performs chunk-level AI-text probability analysis to present interpretable indicators. It also supports on-demand online source verification, pairwise similarity analysis using Jaccard similarity, TF–IDF cosine similarity, and weighted token similarity, as well as PDF and DOCX metadata forensics. The generated reports, similarity matrices, clusters, heatmaps, and metadata records enable teachers to examine suspicious patterns with supporting evidence instead of relying on a single score.

A key contribution of the project is its evidence-oriented design. AI-text probabilities, source matches, document similarity, and metadata anomalies are presented as indicators for instructor review rather than as automatic proof of plagiarism or AI use. This approach recognizes the limitations of OCR quality, probabilistic classification, online search coverage, and editable document metadata, while preserving human judgement as the final basis for academic decisions.

Overall, OriginalityGuard demonstrates that classroom workflows, AI-text indicators, plagiarism-related similarity analysis, OCR, source verification, and document forensics can be integrated into one practical platform. The completed system provides a useful foundation for supporting academic integrity by helping educators prioritize submissions for review, investigate possible concerns systematically, and make informed decisions using transparent analytical evidence.

7. Limitations and Future Enhancements

7.1 Limitations

This project is designed as an originality-assistance system that provides evidence for instructor review; it does not automatically determine whether plagiarism or AI use has occurred. The AI-text analysis module produces probability-based indicators, and these results may be affected by the characteristics of the trained model, writing style, document length, and the continuous evolution of generative AI systems. Therefore, AI-text probability should be interpreted together with academic context and other available evidence.

The accuracy of the text-extraction pipeline depends on the quality of submitted documents. Although the system supports digital text extraction and OCR for scanned PDFs and images, low-resolution scans, noise, poor contrast, complex layouts, mathematical content, and unclear handwriting can reduce extraction quality. Since the extracted text is used for AI analysis, source verification, and similarity comparison, extraction errors may influence the final report.

Online source verification is limited by the availability, accessibility, and indexing of online content. Search engines and external services may not return every relevant source, and a detected web-page similarity does not necessarily establish that a student copied from that source. Likewise, the absence of a matched source does not prove that the content is original. Source-verification results should therefore be treated as possible evidence rather than a final conclusion.

Finally, the current implementation uses in-process background processing for potentially slow source-verification tasks. While this approach is suitable for development and moderate workloads, it may not provide sufficient reliability, monitoring, and scalability for a large institutional deployment with many simultaneous submissions.

7.2 Future Enhancements

Future work can improve the AI-text detector using larger and more diverse validated datasets. Evaluation using precision, recall, F1-score, OCR error rate, and processing time would provide a clearer measure of system performance.

Multilingual support like Nepali, can be added to make the system more suitable for local academic institutions. Embedding-based semantic similarity can also improve the detection of paraphrased content.

The current background processing can be replaced with Celery and Redis for scalable asyn-

chronous analysis, task retries, progress tracking, and completion notifications. Downloadable reports can also be included for formal academic review.

Additional features such as rubric-based grading, teacher comments, audit logs, submission-history tracking, and examiner analytics can improve usability, assessment transparency, and accountability.

Security and deployment can be strengthened through secure cloud storage, stricter file validation, malware scanning, HTTPS, environment-based secret management, restricted CORS policies, and regular security monitoring.

Appendices

This section is reserved for supplementary material that is not central to the main body of the report, such as program source code listings, raw dataset samples, or additional supporting figures. Appendix content will be added as it becomes available in the final phase of the project.

References

[1] A. Schulz et al. Hierarchical stylometric analysis for authorial intent identification. *Journal of Academic Integrity Research*, 2024.

[2] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[3] B. Guo et al. Detecting synthetic text via perplexity and burstiness metrics in LLMs. In *International Conference on Machine Learning Applications*, 2025.

[4] J. Smith and R. Jones. Robust document digitization: Integrating OCR and HTR for academic forensics. *Journal of Digital Archiving and Document Processing*, 2023.

[5] Y. Wang et al. Digital forensic frameworks for metadata integrity in OOXML and PDF documents. *IEEE Transactions on Information Forensics and Security*, 2025.